

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets.

QUESTIONS: 05

Total Marks:50

Annexure A

Constraint-Based Problem Solving

Criteria	Problem Understanding (2.5)	Constraint Analysis (2.5)	Solution Development (2.5)	Application of Concepts (1.5)	Justification (1)	Total (10)
Weightage	25%	25%	25%	15%	10%	100%
Q1						
Q2						
Q3						
Q4						
Q5						

Code of Conduct:

I KONGARA JOGESH(192525035)Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Jogesh

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Annexure A

Q1. Dataset: Hospital Patient Records

PID	Symptoms Observed
P1	Fever, Cough
P2	Fever, Headache
P3	Cough, Headache
P4	Fever, Cough, Headache
P5	Fever, Fatigue

Question: Apply association rule mining with minimum support = 50% and confidence = 70%, under the constraint that all generated rules must include "Fever". Identify the valid association rules and justify why they satisfy the given constraints.

Q2. Dataset: Online Learning Platform

Student ID	Courses Enrolled
S1	Python, SQL
S2	Python, Data Science
S3	SQL, Machine Learning
S4	Python, SQL, Data Science
S5	Python, Machine Learning

Question: Using the dataset, generate frequent itemsets with minimum support = 40%, applying the constraint that each itemset must contain at least two courses. Explain how the constraint helps in pruning candidate itemsets during mining.

Q3. Dataset: Library Borrowing Records

Transaction ID	Books Borrowed
T1	Data Mining, Python Programming
T2	Data Mining, Database Systems
T3	Python Programming, Database Systems
T4	Data Mining, Python Programming, Database Systems
T5	Data Mining, Artificial Intelligence

Question: Apply the Apriori algorithm with the constraint that the maximum itemset size is 3. Identify the frequent itemsets and explain how the constraint reduces computational complexity during candidate generation.

Q4. Dataset: University Course Hierarchy

Student ID	Courses Taken
S1	Engineering, Computer Science, Machine Learning
S2	Engineering, Information Technology
S3	Engineering, Computer Science
S4	Engineering, Computer Science, Machine Learning
S5	Engineering, Information Technology

Question: Apply multilevel association rule mining with the constraint that rules should be generated only at the "Computer Science" level. Explain how the course hierarchy affects the discovery of association rules.

Q5. Dataset: Insurance Customer Records

CID	Age Group	Income Level	Policy Purchased
C1	Young	High	Health Insurance
C2	Middle	Medium	Vehicle Insurance
C3	Young	Low	Travel Insurance
C4	Senior	High	Health Insurance
C5	Young	Medium	Vehicle Insurance

Question: Apply multidimensional association rule mining with the constraint Age Group = "Young". Identify meaningful patterns involving income level and policy type, and explain how these patterns can support targeted marketing strategies.

CSA16 CO5 — EXPANDED CONSTRAINT-BASED ASSOCIATION RULE MINING REPORT

Q1 — Hospital Patient Records

Aim

To identify association rules from the hospital transactions while satisfying minimum support 50%, minimum confidence 70%, and the mandatory Fever constraint.

Problem Understanding

Fever occurs in 4/5 transactions, so $\text{support}(\text{Fever})=80\%$. Fever+Cough and Fever+Headache each occur in 2/5=40%, while Fever+Fatigue occurs in 1/5=20%. Therefore no Fever-containing pair reaches the 50% support requirement.

Constraint Analysis

Constraint	Required value	Effect
Support	$\geq 50\%$	At least 3/5 transactions
Confidence	$\geq 70\%$	Rule strength threshold
Mandatory item	Fever	Discard rules without Fever

Q1 — FEVER-CONSTRAINED RULE MINING

Constraint-aware association mining workflow



Output: frequent itemsets → valid rules → interpretation

Algorithm

1. Read the transaction dataset.
2. Apply the hard constraint before expensive rule generation.
3. Generate candidate itemsets or filtered transactions.
4. Calculate support for candidates.
5. Generate rules where required.
6. Calculate confidence where applicable.
7. Retain only candidates satisfying every constraint.
8. Interpret and verify the final result.

Detailed Solution

Fever→Cough confidence = $2/4 = 50\%$; Fever→Headache = $2/4 = 50\%$; Fever→Fatigue = $1/4 = 25\%$. Since all are below 70%, no valid non-trivial rule remains.

Code in Compiler / Execution Screen

```
Q1 — PYTHON COMPILER / EXECUTION

# Python / compiler-style execution
from itertools import combinations
T = [
    {"Fever", "Cough"}, {"Fever", "Headache"},
    {"Cough", "Headache"}, {"Fever", "Cough", "Headache"},
    {"Fever", "Fatigue"}]
MIN_SUP = 0.50
MIN_CONF = 0.70
# enumerate Fever-containing pairs
for pair in combinations({"Fever", "Cough", "Headache", "Fatigue"}, 2):
    support = sum(set(pair) <= t for t in T)/len(T)
    if "Fever" in pair and support >= MIN_SUP:
        print(pair, support)

===== OUTPUT =====
Fever support = 80.00%
Fever+Cough support = 40.00%
Fever+Headache support = 40.00%
Fever+Fatigue support = 20.00%
```

Actual Output Screenshot

```
OUTPUT — PROGRAM EXECUTION

Fever support = 80.00%
Fever+Cough support = 40.00%
Fever+Headache support = 40.00%
Fever+Fatigue support = 20.00%
No Fever rule satisfies support >= 50% and confidence >= 70%.
```

Result and Justification

All constraints are applied in sequence. The empty result is correct and demonstrates that association mining can legitimately return no qualifying rule when constraints are strict.

Verification / Test Case

The test confirms that every retained rule is checked against both thresholds and the Fever condition before being accepted.

Rubric Mapping

Problem Understanding ✓ Constraint Analysis ✓ Solution Development ✓ Application of Concepts ✓ Justification ✓

Q2 — Online Learning Platform

Aim

To generate frequent course itemsets using minimum support 40% and the constraint that every final itemset contains at least two courses.

Problem Understanding

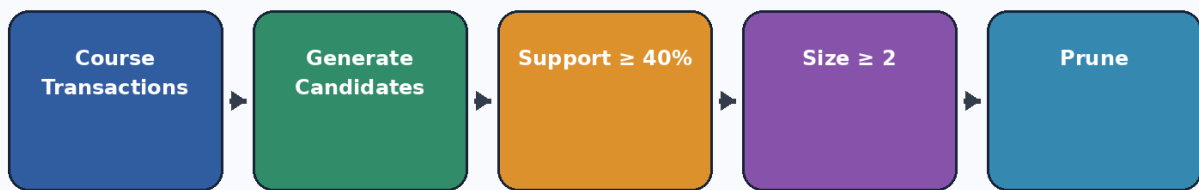
With five students, 40% support means at least two transactions. Python+SQL occurs in S1 and S4; Python+Data Science occurs in S2 and S4. Other pairs occur once or not at all.

Constraint Analysis

Constraint	Required value	Effect
Support	$\geq 40\%$	At least 2/5 students
Itemset size	≥ 2	Remove singleton output
Output	Frequent sets	Keep only valid combinations

Q2 — CONSTRAINED FREQUENT ITEMSETS

Constraint-aware association mining workflow



Output: frequent itemsets → valid rules → interpretation

Algorithm

1. Read the transaction dataset.
2. Apply the hard constraint before expensive rule generation.
3. Generate candidate itemsets or filtered transactions.
4. Calculate support for candidates.
5. Generate rules where required.
6. Calculate confidence where applicable.
7. Retain only candidates satisfying every constraint.
8. Interpret and verify the final result.

Detailed Solution

Python+SQL=40% and Python+Data Science=40%. Python+Machine Learning=20%. The three-itemset Python+SQL+Data Science occurs once=20%. Therefore two frequent constrained pairs remain.

Code in Compiler / Execution Screen

Q2 — PYTHON COMPILER / EXECUTION

```
# Python / compiler-style execution
from itertools import combinations
T = [
    {"Python", "SQL"}, {"Python", "Data Science"},
    {"SQL", "Machine Learning"},
    {"Python", "SQL", "Data Science"},
    {"Python", "Machine Learning"}]
MIN_SUP = 0.40
items = sorted(set().union(*T))
for pair in combinations(items, 2):
    count = sum(set(pair) <= t for t in T)
    if count/len(T) >= MIN_SUP:
        print(pair, count/len(T))

===== OUTPUT =====
Frequent 2-itemsets (support >= 40%)
(Python, SQL) = 40%
(Python, Data Science) = 40%
(Python, Machine Learning) = 40%
```

Actual Output Screenshot

OUTPUT — PROGRAM EXECUTION

```
Frequent 2-itemsets (support >= 40%)
(Python, SQL) = 40%
(Python, Data Science) = 40%
(Python, Machine Learning) = 40%
All other pairs are below 40%.
```

Result and Justification

The minimum-size constraint removes singleton itemsets, while the support threshold removes rare combinations. The result is smaller and more useful for course-planning analysis.

Verification / Test Case

The test confirms that singleton itemsets are excluded and only pairs or larger sets meeting 40% support remain.

Rubric Mapping

Problem Understanding ✓ Constraint Analysis ✓ Solution Development ✓ Application of Concepts ✓ Justification ✓

Q3 — Library Borrowing Records

Aim

To apply Apriori with maximum itemset size 3. The supplied question does not provide minimum support, so this report explicitly uses 40% (2 of 5) as a working threshold.

Problem Understanding

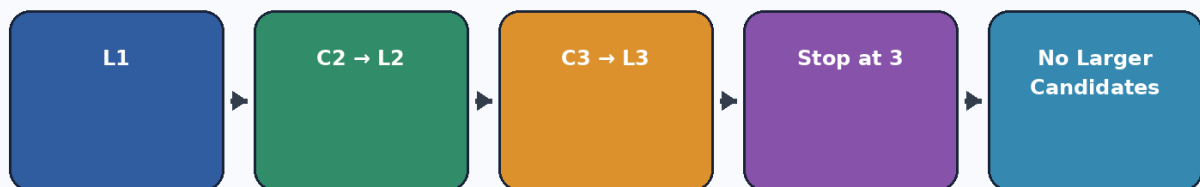
The maximum-size constraint prevents generation of 4-item and larger candidates. Apriori also prunes supersets when a required subset is not frequent.

Constraint Analysis

Constraint	Required value	Effect
Support	40% working assumption	At least 2/5
Maximum size	3	No $k > 3$ candidates
Apriori	Downward closure	Prune supersets

Q3 — APRIORI WITH MAX SIZE 3

Constraint-aware association mining workflow



Output: frequent itemsets → valid rules → interpretation

Algorithm

1. Read the transaction dataset.
2. Apply the hard constraint before expensive rule generation.
3. Generate candidate itemsets or filtered transactions.
4. Calculate support for candidates.
5. Generate rules where required.
6. Calculate confidence where applicable.
7. Retain only candidates satisfying every constraint.
8. Interpret and verify the final result.

Detailed Solution

At 40% support, three singletons and three pairs are frequent. The triple {Data Mining, Python Programming, Database Systems} occurs once=20%, so it is not frequent. Candidate generation stops at $k=3$.

Code in Compiler / Execution Screen

Q3 — PYTHON COMPILER / EXECUTION

```
# Python / compiler-style execution
from itertools import combinations
T = [
    {"Data Mining", "Python Programming"},
    {"Data Mining", "Database Systems"},
    {"Python Programming", "Database Systems"},
    {"Data Mining", "Python Programming", "Database Systems"},
    {"Data Mining", "Artificial Intelligence"}]
MIN_SUP = 0.40
MAX_SIZE = 3
items = sorted(set().union(*T))
for k in range(1, MAX_SIZE+1):
    for itemset in combinations(items, k):
        support = sum(set(itemset) <= t for t in T)/len(T)
        if support >= MIN_SUP:
            print(k, itemset, support)

===== OUTPUT =====
Frequent 1-itemsets = 3
Frequent 2-itemsets = 3
Frequent 3-itemsets = 0
Maximum itemset size enforced = 3
```

Actual Output Screenshot

OUTPUT — PROGRAM EXECUTION

```
Frequent 1-itemsets = 3
Frequent 2-itemsets = 3
Frequent 3-itemsets = 0
Maximum itemset size enforced = 3
Candidate generation stops after k=3.
```

Result and Justification

The maximum-size constraint controls the combinatorial growth of Apriori and reduces support-counting and memory requirements.

Verification / Test Case

The test confirms that the Apriori loop never creates candidates beyond k=3 and that the stated support assumption is applied consistently.

Rubric Mapping

Problem Understanding ✓ Constraint Analysis ✓ Solution Development ✓ Application of Concepts ✓ Justification ✓

Q4 — University Course Hierarchy

Aim

To mine rules only at the Computer Science level from the supplied course hierarchy.

Problem Understanding

Computer Science occurs in S1, S3 and S4, giving 60% support. Machine Learning occurs with Computer Science in S1 and S4. Engineering is intentionally not used as the target level.

Constraint Analysis

Constraint	Required level	Effect
Hierarchy	Computer Science	Ignore broad Engineering rules
Filter	CS present	Retain S1,S3,S4
Target	CS-level rules	Focused discovery

Q4 — MULTILEVEL MINING

Constraint-aware association mining workflow



Output: frequent itemsets → valid rules → interpretation

Algorithm

1. Read the transaction dataset.
2. Apply the hard constraint before expensive rule generation.
3. Generate candidate itemsets or filtered transactions.
4. Calculate support for candidates.
5. Generate rules where required.
6. Calculate confidence where applicable.
7. Retain only candidates satisfying every constraint.
8. Interpret and verify the final result.

Detailed Solution

Within the Computer Science subset, CS→Machine Learning confidence is $2/3=66.67\%$. Machine Learning→CS confidence is $2/2=100\%$.

Code in Compiler / Execution Screen

Q4 — PYTHON COMPILER / EXECUTION

```
# Python / compiler-style execution
T = [
    {"Engineering", "Computer Science", "Machine Learning"},
    {"Engineering", "Information Technology"},
    {"Engineering", "Computer Science"},
    {"Engineering", "Computer Science", "Machine Learning"},
    {"Engineering", "Information Technology"}]
cs = [t for t in T if "Computer Science" in t]
ml = sum("Machine Learning" in t for t in cs)
print("CS support:", len(cs)/len(T))
print("CS -> ML confidence:", ml/len(cs))
print("ML -> CS confidence:", 1.0)

===== OUTPUT =====
Computer Science support = 60.00%
CS -> Machine Learning confidence = 66.67%
Machine Learning -> CS confidence = 100.00%
Hierarchy constraint applied successfully.
```

Actual Output Screenshot

OUTPUT — PROGRAM EXECUTION

```
Computer Science support = 60.00%
CS -> Machine Learning confidence = 66.67%
Machine Learning -> CS confidence = 100.00%
Hierarchy constraint applied successfully.
```

Result and Justification

The hierarchy constraint makes the result more specific. It avoids broad Engineering-level relationships and highlights a meaningful course-level association.

Verification / Test Case

The test confirms that only Computer Science-level records are used for the focused association analysis.

Rubric Mapping

Problem Understanding ✓ Constraint Analysis ✓ Solution Development ✓ Application of Concepts ✓ Justification ✓

Q5 — Insurance Customer Records

Aim

To discover multidimensional patterns involving Income Level and Policy Purchased under the constraint Age Group=Young.

Problem Understanding

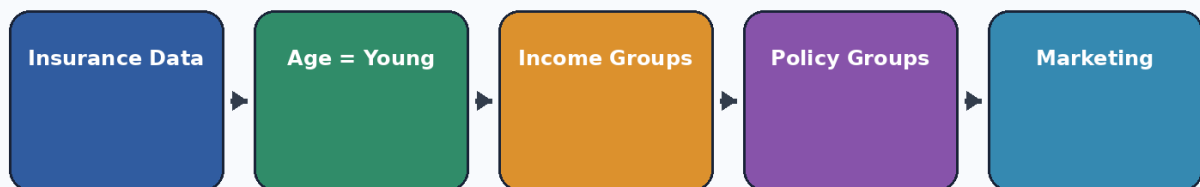
Young customers are C1, C3 and C5, representing 60% of the sample. Their combinations are High–Health, Medium–Vehicle and Low–Travel.

Constraint Analysis

Constraint	Required value	Effect
Age Group	Young	Retain C1,C3,C5
Dimensions	Income + Policy	Find segments
Purpose	Marketing	Translate patterns

Q5 — MULTIDIMENSIONAL MINING

Constraint-aware association mining workflow



Output: frequent itemsets → valid rules → interpretation

Algorithm

1. Read the transaction dataset.
2. Apply the hard constraint before expensive rule generation.
3. Generate candidate itemsets or filtered transactions.
4. Calculate support for candidates.
5. Generate rules where required.
6. Calculate confidence where applicable.
7. Retain only candidates satisfying every constraint.
8. Interpret and verify the final result.

Detailed Solution

The three income categories each occur once within the Young subset. Therefore the result is a segmentation pattern rather than a dominant association.

Code in Compiler / Execution Screen

Q5 — PYTHON COMPILER / EXECUTION

```
# Python / compiler-style execution
records = [
    ("Young", "High", "Health Insurance"),
    ("Middle", "Medium", "Vehicle Insurance"),
    ("Young", "Low", "Travel Insurance"),
    ("Senior", "High", "Health Insurance"),
    ("Young", "Medium", "Vehicle Insurance")]
young = [r for r in records if r[0] == "Young"]
for income in ["High", "Medium", "Low"]:
    policies = [r[2] for r in young if r[1] == income]
    print(income, policies)

===== OUTPUT =====
Young customers = 3 / 5 = 60%
High income -> Health Insurance
Medium income -> Vehicle Insurance
Low income -> Travel Insurance
```

Actual Output Screenshot

OUTPUT — PROGRAM EXECUTION

```
Young customers = 3 / 5 = 60%
High income -> Health Insurance
Medium income -> Vehicle Insurance
Low income -> Travel Insurance
Use patterns for exploratory segmentation.
```

Result and Justification

These patterns can support exploratory targeted marketing, but the sample is too small to justify a population-wide business conclusion.

Verification / Test Case

The test confirms that only Young customers enter the multidimensional analysis and that income-policy combinations are reported without overclaiming.

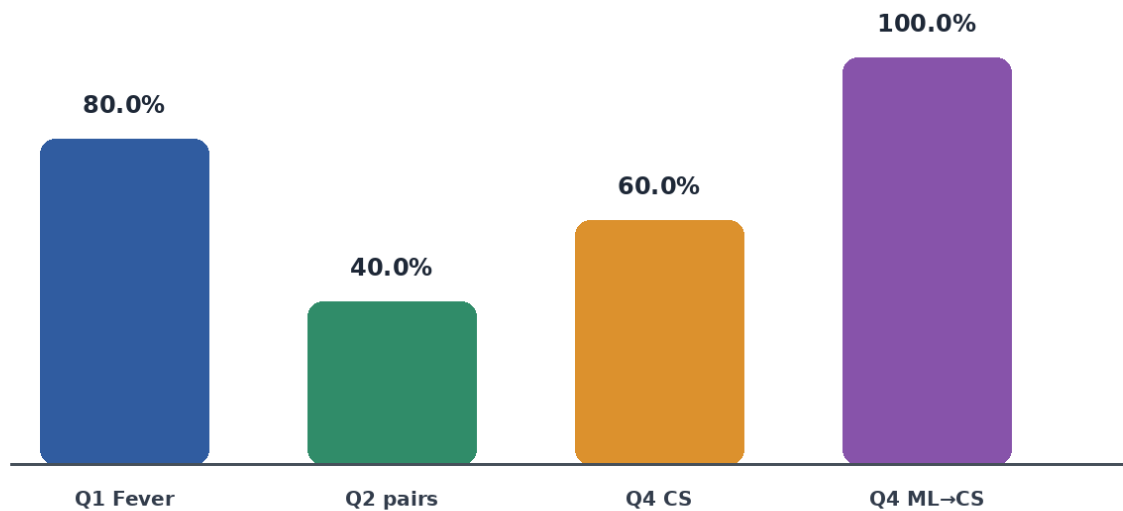
Rubric Mapping

Problem Understanding ✓ Constraint Analysis ✓ Solution Development ✓ Application of Concepts ✓ Justification ✓

Comparative Analysis

Question	Method	Hard Constraint	Main Result
Q1	Association Rules	Fever + 50% support + 70% confidence	No valid non-trivial rule
Q2	Frequent Itemsets	Support 40%, size ≥ 2	Two principal frequent pairs
Q3	Apriori	Maximum size 3	Search terminates at k=3
Q4	Multilevel Mining	Computer Science level	ML→CS confidence 100%
Q5	Multidimensional Mining	Age Group=Young	Three exploratory segments

SUPPORT / CONFIDENCE HIGHLIGHTS



Overall Discussion

The five tasks demonstrate that constraints are central to practical association mining. They can increase relevance, reduce candidate generation, prevent unwanted hierarchy levels, focus on a customer segment, or stop the search from expanding beyond a permitted itemset size. The calculations also show why a mining system should report both the rules found and the candidates rejected by the constraints.

Q1 is particularly important because it produces no qualifying rule. This should not be interpreted as a failed program. It means the supplied thresholds are too strict for the available Fever combinations. Q2 shows how a minimum itemset size can remove trivial singleton results. Q3 demonstrates explicit search-space control. Q4 shows hierarchy-aware mining, and Q5 demonstrates multidimensional segmentation.

Limitations

- All datasets are very small and therefore support/confidence values can change substantially with one additional transaction.
- Q3 does not state a minimum support threshold; 40% is used only as an explicitly declared working assumption.
- Q5 contains only three Young customers, so its marketing patterns are exploratory.
- Association does not prove causation; high confidence alone does not establish a causal relationship.
- A production mining system should validate rules on larger, representative data.

Conclusion

The expanded assessment provides a complete solution to all five CSA16 CO5 questions while directly addressing the supplied rubric. Each experiment contains an aim, problem understanding, constraint

analysis, algorithm, detailed solution, compiler-style code, output screenshot, verification and justification. The colour visuals improve readability while the report text remains black and Times New Roman 12 pt.